NLDL Programming

MCA-575

Assignment – 04

BY

HIMANSHU HEDA (24225013)

SUBMITTED TO

Dr. Manjula Shannhog

SCHOOL OF SCIENCES

2025-26

Importing Libraries:

```
import numpy as np
import pandas as pd
```

Import Dataset:

```
df = pd.read_csv('./Dataset/customer.csv')
```

Shape:

```
df.shape
```

Head:

```
df.head()
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	1	22	Female	25	14	4	27	Basic	Monthly	598	9	1
1	2	41	Female	28	28	7	13	Standard	Monthly	584	20	0
2	3	47	Male	27	10	2	29	Premium	Annual	757	21	0
3	4	35	Male	9	12	5	17	Premium	Quarterly	232	18	0
4	5	53	Female	58	24	9	2	Standard	Annual	533	18	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         64374 non-null  int64
 1   Age                64374 non-null  int64
 2   Gender             64374 non-null  object
 3   Tenure             64374 non-null  int64
 4   Usage Frequency    64374 non-null  int64
 5   Support Calls      64374 non-null  int64
 6   Payment Delay      64374 non-null  int64
 7   Subscription Type  64374 non-null  object
 8   Contract Length    64374 non-null  object
 9   Total Spend        64374 non-null  int64
 10  Last Interaction   64374 non-null  int64
 11  Churn              64374 non-null  int64
dtypes: int64(9), object(3)
memory usage: 5.9+ MB
```

```python
df.duplicated().sum()
```

```
np.int64(0)
```

```python
df['Churn'].value_counts()
```

```
Churn
0    33881
1    30493
Name: count, dtype: int64
```

```python
df['Gender'].value_counts()
```

```
Gender
Female    34353
Male      30021
Name: count, dtype: int64
```

```python
df.drop('CustomerID',axis=1,inplace=True)
```

```python
df.head()
```

	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	22	Female	25	14	4	27	Basic	Monthly	598	9	1
1	41	Female	28	28	7	13	Standard	Monthly	584	20	0
2	47	Male	27	10	2	29	Premium	Annual	757	21	0
3	35	Male	9	12	5	17	Premium	Quarterly	232	18	0
4	53	Female	58	24	9	2	Standard	Annual	533	18	0

Unique:

```python
df['Gender'].unique()
```

```
array(['Female', 'Male'], dtype=object)
```

```python
df['Subscription Type'].unique()
```

```
array(['Basic', 'Standard', 'Premium'], dtype=object)
```

```python
df['Contract Length'].unique()
```

```
array(['Monthly', 'Annual', 'Quarterly'], dtype=object)
```

```python
df=pd.get_dummies(df, columns=['Gender', 'Subscription Type','Contract Length'], drop_first=True, dtype=int)
```

```python
df.head()
```

	Age	Tenure	Usage Frequency	Support Calls	Payment Delay	Total Spend	Last Interaction	Churn	Gender_Male	Subscription Type_Premium	Subscription Type_Standard	Contract Length_Monthly	Contract Length_Quarterly
0	22	25	14	4	27	598	9	1	0	0	0	1	0
1	41	28	28	7	13	584	20	0	0	0	1	1	0
2	47	27	10	2	29	757	21	0	1	1	0	0	0
3	35	9	12	5	17	232	18	0	1	1	0	0	1
4	53	58	24	9	2	533	18	0	0	0	1	0	0

```python
df.info()
```
✓ 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 13 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Age                         64374 non-null  int64
 1   Tenure                      64374 non-null  int64
 2   Usage Frequency             64374 non-null  int64
 3   Support Calls               64374 non-null  int64
 4   Payment Delay               64374 non-null  int64
 5   Total Spend                 64374 non-null  int64
 6   Last Interaction            64374 non-null  int64
 7   Churn                       64374 non-null  int64
 8   Gender_Male                 64374 non-null  int64
 9   Subscription Type_Premium   64374 non-null  int64
 10  Subscription Type_Standard  64374 non-null  int64
 11  Contract Length_Monthly     64374 non-null  int64
 12  Contract Length_Quarterly   64374 non-null  int64
dtypes: int64(13)
memory usage: 6.4 MB
```

```python
x=df.drop('Churn',axis=1)
```
✓ 0.0s Python

```python
x.head()
```
✓ 0.0s Python

	Age	Tenure	Usage Frequency	Support Calls	Payment Delay	Total Spend	Last Interaction	Gender_Male	Subscription Type_Premium	Subscription Type_Standard	Contract Length_Monthly	Contract Length_Quarterly
0	22	25	14	4	27	598	9	0	0	0	1	0
1	41	28	28	7	13	584	20	0	0	1	1	0
2	47	27	10	2	29	757	21	1	1	0	0	0
3	35	9	12	5	17	232	18	1	1	0	0	1
4	53	58	24	9	2	533	18	0	0	1	0	0

Importing TensorFlow for using keras , Sequential and Dense

```python
x.shape
```
✓ 0.0s Python

```
(64374, 12)
```

```python
y=df['Churn']
```
✓ 0.0s Python

```python
y.head()
```
✓ 0.0s Python

```
0    1
1    0
2    0
3    0
4    0
Name: Churn, dtype: int64
```

```python
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(x,y,test_size=0.2, random_state=1)
```
✓ 4.1s Python

```python
import tensorflow
from tensorflow import keras
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```
✓ 7.2s Python

```python
model=Sequential()
```
✓ 0.0s Python

```python
model.add(Dense(3,activation='sigmoid', input_dim=12))
model.add(Dense(1, activation='sigmoid'))
```
✓ 0.1s Python

```
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
```

Model Summary & Using Sequential:

```
model.summary()
```

✓ 0.0s Python

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 3)	39
dense_1 (Dense)	(None, 1)	4

```
Total params: 43 (172.00 B)

Trainable params: 43 (172.00 B)

Non-trainable params: 0 (0.00 B)
```

```
model.compile(loss='binary_crossentropy', optimizer='Adam')
```

✓ 0.0s Python

Model with 20 Epoch:

```
model.fit(xtrain,ytrain,epochs=20)
```

✓ 1m 32.1s Python

```
Epoch 1/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - loss: 0.6920
Epoch 2/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.6000
Epoch 3/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - loss: 0.5217
Epoch 4/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.5050
Epoch 5/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.4955
Epoch 6/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - loss: 0.4844
Epoch 7/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.4729
Epoch 8/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.4608
Epoch 9/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.4525
Epoch 10/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - loss: 0.4455
Epoch 11/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 0.4398
Epoch 12/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 0.4351
Epoch 13/20
...
Epoch 19/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - loss: 0.4114
Epoch 20/20
1610/1610 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - loss: 0.4092
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
<keras.src.callbacks.history.History at 0x1e347c9a510>
```

```
model.layers[0].get_weights()
```

✓ 0.0s Python

```
[array([[ 0.24747407,  0.5484759 , -0.01030802],
        [ 0.5034012 , -0.56922996, -0.03204678],
        [ 0.5322016 ,  0.60176116,  0.05271243],
        [ 0.45412868, -0.15800229, -0.29941306],
        [ 0.5783252 ,  0.22603565, -0.1961106 ],
        [ 0.14990705,  0.47634274,  0.00136405],
        [ 0.36738688,  0.6228965 ,  0.0064075 ],
        [ 0.31013888, -0.10547173,  1.1184093 ],
        [ 0.40496856, -0.6299548 ,  0.19054295],
        [ 0.59969944, -0.17263186,  0.13736075],
        [-0.3646611 , -0.31151456, -0.16942868],
        [ 0.5950776 ,  0.5580589 ,  0.2572021 ]], dtype=float32),
 array([0.        , 0.        , 4.3123455], dtype=float32)]
```

```
model.layers[1].get_weights()
```

✓ 0.0s Python

```
[array([[ 0.3622426],
        [ 1.2039508],
        [-5.5814004]], dtype=float32),
 array([1.0140944], dtype=float32)]
```

```
model.predict(xtest)
```

✓ 1.6s Python

```
403/403 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

array([[0.56197524],
       [0.7966516 ],
       [0.8543299 ],
       ...,
       [0.9142713 ],
       [0.09655276],
       [0.05410805]], shape=(12875, 1), dtype=float32)
```

```
xtest_prob=model.predict(xtest)
```

✓ 1.2s Python

```
403/403 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

```
ypred=np.where(xtest_prob>0.5,1,0)
```

✓ 0.0s Python

```
ypred
```

✓ 0.0s Python

```
array([[1],
       [1],
       [1],
       ...,
       [1],
       [0],
       [0]], shape=(12875, 1))
```

```
from sklearn.metrics import accuracy_score
accuracy_score(ytest,ypred)
```

✓ 0.0s Python

```
0.8276504854368932
```

Modified

```
model1=Sequential()
```
✓ 0.0s Python

```
model1.add(Dense(12,activation='relu', input_dim=12))
model1.add(Dense(12, activation='relu'))
model1.add(Dense(1, activation='sigmoid'))
```
✓ 0.0s Python

```
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model1.summary()
```
✓ 0.0s Python

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 12)	156
dense_3 (Dense)	(None, 12)	156
dense_4 (Dense)	(None, 1)	13

Total params: 325 (1.27 KB)

Trainable params: 325 (1.27 KB)

Non-trainable params: 0 (0.00 B)

Model Accuracy with 20 Epoch:

```
model1.compile(loss='binary_crossentropy', optimizer='Adam', metrics=['accuracy'])
```
✓ 0.0s Python

```
history=model1.fit(xtrain,ytrain, epochs=20, validation_split=0.2)
```
✓ 1m 59.1s Python

```
Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.7245 - loss: 0.9794 - val_accuracy: 0.7844 - val_loss: 0.4695
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7791 - loss: 0.4661 - val_accuracy: 0.7975 - val_loss: 0.4376
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8039 - loss: 0.4270 - val_accuracy: 0.8158 - val_loss: 0.3983
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8069 - loss: 0.4220 - val_accuracy: 0.8242 - val_loss: 0.3911
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8163 - loss: 0.4017 - val_accuracy: 0.8221 - val_loss: 0.3774
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8257 - loss: 0.3764 - val_accuracy: 0.8384 - val_loss: 0.3479
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8339 - loss: 0.3598 - val_accuracy: 0.8514 - val_loss: 0.3334
Epoch 8/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8410 - loss: 0.3440 - val_accuracy: 0.8548 - val_loss: 0.3222
Epoch 9/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8475 - loss: 0.3347 - val_accuracy: 0.8571 - val_loss: 0.3108
Epoch 10/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8504 - loss: 0.3265 - val_accuracy: 0.8513 - val_loss: 0.3277
Epoch 11/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8524 - loss: 0.3215 - val_accuracy: 0.8582 - val_loss: 0.3074
Epoch 12/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8524 - loss: 0.3197 - val_accuracy: 0.8250 - val_loss: 0.3886
Epoch 13/20
...
Epoch 19/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 11s 8ms/step - accuracy: 0.8819 - loss: 0.2635 - val_accuracy: 0.8925 - val_loss: 0.2513
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.8894 - loss: 0.2497 - val_accuracy: 0.8981 - val_loss: 0.2333
```
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

History:

```python
history.history
```

✓ 0.0s Python

```
{'accuracy': [0.7245078682899475,
  0.7791451215744019,
  0.8039273023605347,
  0.8068885207176208,
  0.8163304924964905,
  0.8256511092185974,
  0.8339037299156189,
  0.8409912586212158,
  0.8474720120429993,
  0.8503847122192383,
  0.8524478673934937,
  0.8524478673934937,
  0.8559430837631226,
  0.8570839166641235,
  0.8568654656410217,
  0.858831524848938,
  0.8604577779769897,
  0.8675938844680786,
  0.8819146156311035,
  0.8893662691116333],
 'loss': [0.9793554544448853,
  0.4661440849304199,
  0.42701810598373413,
  0.4219523072242737,
  0.4016907215118408,
...
  0.3085147440433502,
  0.2903953790664673,
  0.27636921405792236,
  0.2513231635093689,
  0.23333494365215302]}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Model Layers:

```python
model1.layers[0].get_weights()
```

✓ 0.0s Python

```
[array([[ 3.03661138e-01, -1.21089697e-01,  2.86132097e-01,
          5.34513779e-02, -1.27409503e-01,  3.47570628e-01,
          1.75568178e-01,  2.83738732e-01, -9.34195146e-02,
         -2.47545809e-01, -2.15495467e-01, -1.45413637e-01],
        [ 2.07149297e-01,  3.75049591e-01, -2.69613951e-01,
          2.49457881e-01,  1.70818925e-01,  1.13568630e-03,
         -3.01587909e-01,  1.69191703e-01, -2.85725184e-02,
         -2.20541954e-01,  1.14585638e-01, -1.55682087e-01],
        [ 1.65632926e-02,  9.47405100e-02, -1.03158462e+00,
          8.73592123e-02, -3.91364023e-02,  3.35201889e-01,
         -1.32469162e-01,  2.24173851e-02,  1.64432788e+00,
          5.26549369e-02,  1.63931608e-01,  1.07286096e-01],
        [-1.35352218e+00,  7.41077662e-02, -5.24353564e-01,
          2.95778334e-01,  2.56276876e-01, -6.18877336e-02,
          1.85530925e+00, -1.82321280e-01, -1.35769904e+00,
         -2.25881308e-01, -7.30607510e-02,  1.41040802e-01],
        [ 1.39139518e-01,  3.97790670e-02, -1.29076317e-01,
         -1.11071146e+00,  3.24396998e-01,  1.23171337e-01,
          1.34695858e-01,  4.37317014e-01, -1.70511425e-01,
         -2.29458869e-01, -2.66830206e-01, -1.85014009e-02],
        [-3.31586972e-02, -3.58837247e-01,  1.26054242e-01,
          9.20074759e-04,  9.01892558e-02, -4.13203925e-01,
          2.58982088e-02, -4.37653847e-02,  5.93897477e-02,
          3.55745740e-02, -4.07632470e-01, -6.73358440e-02],
        [-2.07040831e-01,  6.68512583e-02,  1.79316208e-01,
...
          8.49494785e-02, -4.81387556e-01,  1.63568151e+00,
          9.09123659e-01,  4.58206058e-01, -2.53692269e-01]], dtype=float32),
 array([-0.01184493,  0.        ,  1.6096745 ,  2.483892  , -1.8228875 ,
        -0.08262815, -1.5206857 , -0.26418805,  1.1068952 ,  1.6290597 ,
         0.        ,  0.        ], dtype=float32)]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
model1.layers[1].get_weights()
```

✓ 0.0s Python

```
[array([[ 2.9467326e-02, -3.5192899e-02, -4.1087300e-02, -1.8275855e-01,
         -1.5905468e-01,  5.9526616e-01,  2.6368868e-01, -2.0919466e-01,
         -7.0314065e-02, -2.8348303e-01, -1.0506206e-01, -3.2864243e-01],
        [ 4.7833478e-01, -1.1356950e-01, -1.1160624e-01,  1.7326641e-01,
         -1.4083564e-01,  4.8322415e-01, -1.3830960e-01,  1.0168421e-01,
          4.5195210e-01, -1.7893255e-01, -4.6671498e-01,  4.1355145e-01],
        [-2.8584740e-01, -1.4147358e-01, -3.1238210e-01,  1.8417586e-01,
         -4.9787378e-01,  1.4655565e-01,  1.2125517e-01, -4.8088658e-01,
          9.6764885e-02, -7.3940814e-02,  3.7105584e-01, -2.7337620e-01],
        [ 2.2228388e-01,  3.2727537e-01, -8.7061393e-01, -4.0557277e-01,
          4.1535991e-01, -7.5550240e-01, -8.0056578e-02, -4.7555935e-01,
         -1.8127650e-01, -2.0808388e-01,  4.1482934e-01, -9.2267171e-02],
        [ 2.4250610e-01,  6.3173413e-02,  4.3032828e-01, -2.3621616e-01,
         -2.9907951e-01,  1.3732767e-01,  2.0715518e-01, -4.3636259e-01,
         -2.9336351e-01, -5.8092427e-01, -2.5489569e-01, -2.6900160e-01],
        [ 3.3637667e-01,  2.4758364e-01, -1.6347611e-01,  6.2650271e-02,
          1.0476482e-01, -4.2386055e-03,  3.6232603e-01, -4.2210531e-01,
         -3.5555845e-01, -4.0871012e-01, -4.8638022e-01,  3.2367098e-01],
        [-6.5716738e-01,  1.5402885e-01, -1.8557219e-02,  4.1155264e-01,
         -7.7154346e-02, -7.8903091e-01, -4.1766292e-01,  1.3009433e-01,
          3.5010311e-01, -4.6399707e-01,  8.6663640e-04, -8.0458872e-02],
        [-6.0488973e-02, -1.2021497e-01,  1.6066419e-01,  4.5752069e-01,
          3.6199552e-01, -2.5935543e-01, -1.5612028e-02,  3.4094837e-01,
          1.3762365e-01, -5.1681995e-02,  9.8495513e-02,  5.6606418e-01],
        [-1.2037344e-01,  1.4462607e-02,  1.5445101e-01,  3.8535830e-01,
...
         -1.8330884e-01, -2.7087390e-01,  2.7277207e-01,  4.4711924e-01]],
       dtype=float32),
 array([-0.26211423,  1.559488  , -1.7340856 , -1.5827535 , -0.07378989,
        -0.21034387, -0.10136005, -0.00599488,  1.5891302 , -0.1340936 ,
        -0.44187936,  0.05391507], dtype=float32)]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
xtest_pro=model1.predict(xtest)
```

✓ 2.2s Python

```
403/403 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step
```

```python
ypred=np.where(xtest_pro>0.5,1,0)
```

✓ 0.0s Python

```python
accuracy_score(ytest,ypred)
```

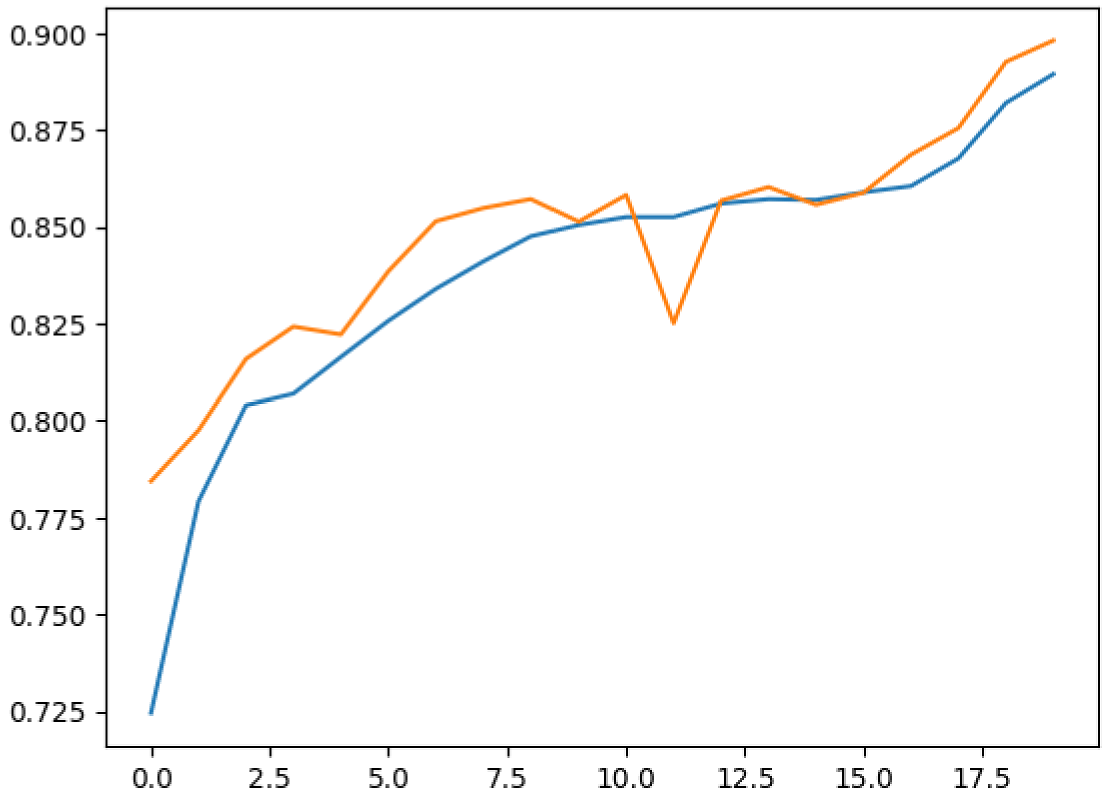
✓ 0.0s Python

```
0.8938252427184467
```

Importing Matplotlib for Graphical Presentation of Accuracies:

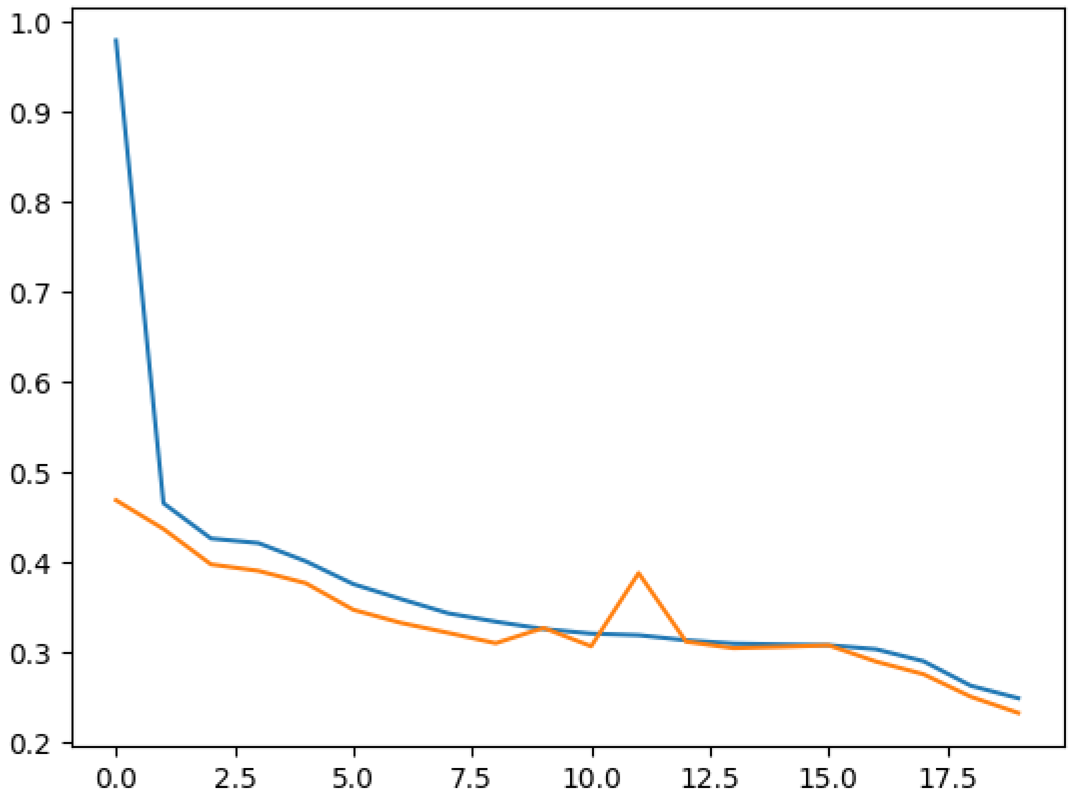
```python
import matplotlib.pyplot as plt
```

```python
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
```



Loss Graph:

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```



Models I have Used are like below

ReLU, Sigmoid, Tanh, Softmax, ELU Activations with Binary Crossentropy and Adam Optimizer.

```
# Model 2: ReLU activation, binary_crossentropy loss
model2 = Sequential()
model2.add(Dense(12, activation='relu', input_dim=12))
model2.add(Dense(12, activation='relu'))
model2.add(Dense(1, activation='sigmoid'))
model2.summary()
model2.compile(loss='binary_crossentropy', optimizer='Adam',
metrics=['accuracy'])
history2 = model2.fit(xtrain, ytrain, epochs=20, validation_split=0.2)

# Model 3: Tanh activation, mean_squared_error loss
model3 = Sequential()
model3.add(Dense(12, activation='tanh', input_dim=12))
model3.add(Dense(12, activation='tanh'))
model3.add(Dense(1, activation='sigmoid'))
model3.summary()
model3.compile(loss='mean_squared_error', optimizer='Adam',
metrics=['accuracy'])
history3 = model3.fit(xtrain, ytrain, epochs=20, validation_split=0.2)

# Model 4: Softmax activation, categorical_crossentropy loss
```

```python
from tensorflow.keras.utils import to_categorical
ytrain_cat = to_categorical(ytrain)
ytest_cat = to_categorical(ytest)
model4 = Sequential()
model4.add(Dense(12, activation='relu', input_dim=12))
model4.add(Dense(12, activation='relu'))
model4.add(Dense(2, activation='softmax'))
model4.summary()
model4.compile(loss='categorical_crossentropy', optimizer='Adam',
metrics=['accuracy'])
history4 = model4.fit(xtrain, ytrain_cat, epochs=20, validation_split=0.2)

# Model 5: ELU activation, hinge loss
model5 = Sequential()
model5.add(Dense(12, activation='elu', input_dim=12))
model5.add(Dense(12, activation='elu'))
model5.add(Dense(1, activation='sigmoid'))
model5.summary()
model5.compile(loss='hinge', optimizer='Adam', metrics=['accuracy'])
history5 = model5.fit(xtrain, ytrain, epochs=20, validation_split=0.2)
```

```
Model: "sequential_9"

┌──────────────────────────┬─────────────────┬────────┐
│ Layer (type)             │ Output Shape    │ Param #│
├──────────────────────────┼─────────────────┼────────┤
│ dense_17 (Dense)         │ (None, 12)      │    156 │
├──────────────────────────┼─────────────────┼────────┤
│ dense_18 (Dense)         │ (None, 12)      │    156 │
├──────────────────────────┼─────────────────┼────────┤
│ dense_19 (Dense)         │ (None, 1)       │     13 │
└──────────────────────────┴─────────────────┴────────┘

 Total params: 325 (1.27 KB)

 Trainable params: 325 (1.27 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7237 - loss: 0.8250 - val_accuracy: 0.7860 - val_loss: 0.4565
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7237 - loss: 0.8250 - val_accuracy: 0.7860 - val_loss: 0.4565
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7871 - loss: 0.4537 - val_accuracy: 0.7329 - val_loss: 0.5400
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7871 - loss: 0.4537 - val_accuracy: 0.7329 - val_loss: 0.5400
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7996 - loss: 0.4356 - val_accuracy: 0.8009 - val_loss: 0.4256
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7996 - loss: 0.4356 - val_accuracy: 0.8009 - val_loss: 0.4256
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8059 - loss: 0.4254 - val_accuracy: 0.8155 - val_loss: 0.4066
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8059 - loss: 0.4254 - val_accuracy: 0.8155 - val_loss: 0.4066
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8127 - loss: 0.4142 - val_accuracy: 0.8163 - val_loss: 0.4055
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8127 - loss: 0.4142 - val_accuracy: 0.8163 - val_loss: 0.4055
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8165 - loss: 0.4107 - val_accuracy: 0.8127 - val_loss: 0.4119
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8165 - loss: 0.4107 - val_accuracy: 0.8127 - val_loss: 0.4119
```

```
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8165 - loss: 0.4107 - val_accuracy: 0.8127 - val_loss: 0.4119
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8165 - loss: 0.4107 - val_accuracy: 0.8127 - val_loss: 0.4119
Epoch 7/20
...
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8322 - loss: 0.3793 - val_accuracy: 0.8304 - val_loss: 0.3730
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8320 - loss: 0.3819 - val_accuracy: 0.8359 - val_loss: 0.3666
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8320 - loss: 0.3819 - val_accuracy: 0.8359 - val_loss: 0.3666
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_10"

Layer (type)	Output Shape	Param #
dense_20 (Dense)	(None, 12)	156
dense_21 (Dense)	(None, 12)	156
dense_22 (Dense)	(None, 1)	13

Total params: 325 (1.27 KB)

Python

Non-trainable params: 0 (0.00 B)

```
Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5789 - loss: 0.2367 - val_accuracy: 0.6928 - val_loss: 0.1954
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5789 - loss: 0.2367 - val_accuracy: 0.6928 - val_loss: 0.1954
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7414 - loss: 0.1772 - val_accuracy: 0.7383 - val_loss: 0.1705
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7414 - loss: 0.1772 - val_accuracy: 0.7383 - val_loss: 0.1705
```

```
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7414 - loss: 0.1772 - val_accuracy: 0.7383 - val_loss: 0.1705
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7414 - loss: 0.1772 - val_accuracy: 0.7383 - val_loss: 0.1705
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7697 - loss: 0.1642 - val_accuracy: 0.7819 - val_loss: 0.1600
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7697 - loss: 0.1642 - val_accuracy: 0.7819 - val_loss: 0.1600
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7838 - loss: 0.1551 - val_accuracy: 0.7852 - val_loss: 0.1541
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7838 - loss: 0.1551 - val_accuracy: 0.7852 - val_loss: 0.1541
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7858 - loss: 0.1531 - val_accuracy: 0.7899 - val_loss: 0.1511
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7858 - loss: 0.1531 - val_accuracy: 0.7899 - val_loss: 0.1511
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7892 - loss: 0.1508 - val_accuracy: 0.7940 - val_loss: 0.1498
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7892 - loss: 0.1508 - val_accuracy: 0.7940 - val_loss: 0.1498
Epoch 7/20
...
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8168 - loss: 0.1343 - val_accuracy: 0.8207 - val_loss: 0.1318
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8140 - loss: 0.1342 - val_accuracy: 0.8213 - val_loss: 0.1326
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8140 - loss: 0.1342 - val_accuracy: 0.8213 - val_loss: 0.1326
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_11"

Layer (type)	Output Shape	Param #
dense_23 (Dense)	(None, 12)	156
dense_24 (Dense)	(None, 12)	156
dense_25 (Dense)	(None, 2)	26

Total params: 338 (1.32 KB)

```
Total params: 338 (1.32 KB)

Trainable params: 338 (1.32 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.7206 - loss: 1.2919 - val_accuracy: 0.7319 - val_loss: 0.5749
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.7206 - loss: 1.2919 - val_accuracy: 0.7319 - val_loss: 0.5749
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7760 - loss: 0.4899 - val_accuracy: 0.7453 - val_loss: 0.5408
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7760 - loss: 0.4899 - val_accuracy: 0.7453 - val_loss: 0.5408
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7808 - loss: 0.4862 - val_accuracy: 0.7847 - val_loss: 0.4704
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7808 - loss: 0.4862 - val_accuracy: 0.7847 - val_loss: 0.4704
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7834 - loss: 0.4840 - val_accuracy: 0.8158 - val_loss: 0.4064
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7834 - loss: 0.4840 - val_accuracy: 0.8158 - val_loss: 0.4064
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7913 - loss: 0.4612 - val_accuracy: 0.7089 - val_loss: 0.6947
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7913 - loss: 0.4612 - val_accuracy: 0.7089 - val_loss: 0.6947
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7912 - loss: 0.4667 - val_accuracy: 0.8134 - val_loss: 0.4157
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7912 - loss: 0.4667 - val_accuracy: 0.8134 - val_loss: 0.4157
Epoch 7/20
...
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8182 - loss: 0.4125 - val_accuracy: 0.8183 - val_loss: 0.3992
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8210 - loss: 0.4046 - val_accuracy: 0.7806 - val_loss: 0.5081
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8210 - loss: 0.4046 - val_accuracy: 0.7806 - val_loss: 0.5081
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```


```
Total params: 338 (1.32 KB)

Trainable params: 338 (1.32 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.7206 - loss: 1.2919 - val_accuracy: 0.7319 - val_loss: 0.5749
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.7206 - loss: 1.2919 - val_accuracy: 0.7319 - val_loss: 0.5749
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7760 - loss: 0.4899 - val_accuracy: 0.7453 - val_loss: 0.5408
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7760 - loss: 0.4899 - val_accuracy: 0.7453 - val_loss: 0.5408
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7808 - loss: 0.4862 - val_accuracy: 0.7847 - val_loss: 0.4704
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7808 - loss: 0.4862 - val_accuracy: 0.7847 - val_loss: 0.4704
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7834 - loss: 0.4840 - val_accuracy: 0.8158 - val_loss: 0.4064
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7834 - loss: 0.4840 - val_accuracy: 0.8158 - val_loss: 0.4064
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7913 - loss: 0.4612 - val_accuracy: 0.7089 - val_loss: 0.6947
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7913 - loss: 0.4612 - val_accuracy: 0.7089 - val_loss: 0.6947
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7912 - loss: 0.4667 - val_accuracy: 0.8134 - val_loss: 0.4157
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7912 - loss: 0.4667 - val_accuracy: 0.8134 - val_loss: 0.4157
Epoch 7/20
...
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8182 - loss: 0.4125 - val_accuracy: 0.8183 - val_loss: 0.3992
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8210 - loss: 0.4046 - val_accuracy: 0.7806 - val_loss: 0.5081
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8210 - loss: 0.4046 - val_accuracy: 0.7806 - val_loss: 0.5081
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
c:\Users\himan\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input_shape`/`input_dim` argume
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 2/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 3/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 4/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 5/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 6/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 7/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 7/20
...
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
Epoch 20/20
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
1288/1288 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4742 - loss: 1.0515 - val_accuracy: 0.4693 - val_loss: 1.0614
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...